



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование

WebSocke
f

WebSocket



Исторический контекст

- Первые веб-приложения реального времени начали появляться в 2000-х годах, пытаясь обеспечить отзывчивый, динамичный и интерактивный опыт конечных пользователей.
- Однако в то время веб в реальном времени был труднодостижим и работал медленнее, чем мы привыкли сегодня
 - Он обеспечивался путем «хакинга» существующих технологий на основе HTTP, которые не были разработаны и оптимизированы для приложений реального времени.
 - Быстро стало очевидно, что необходима лучшая альтернатива.



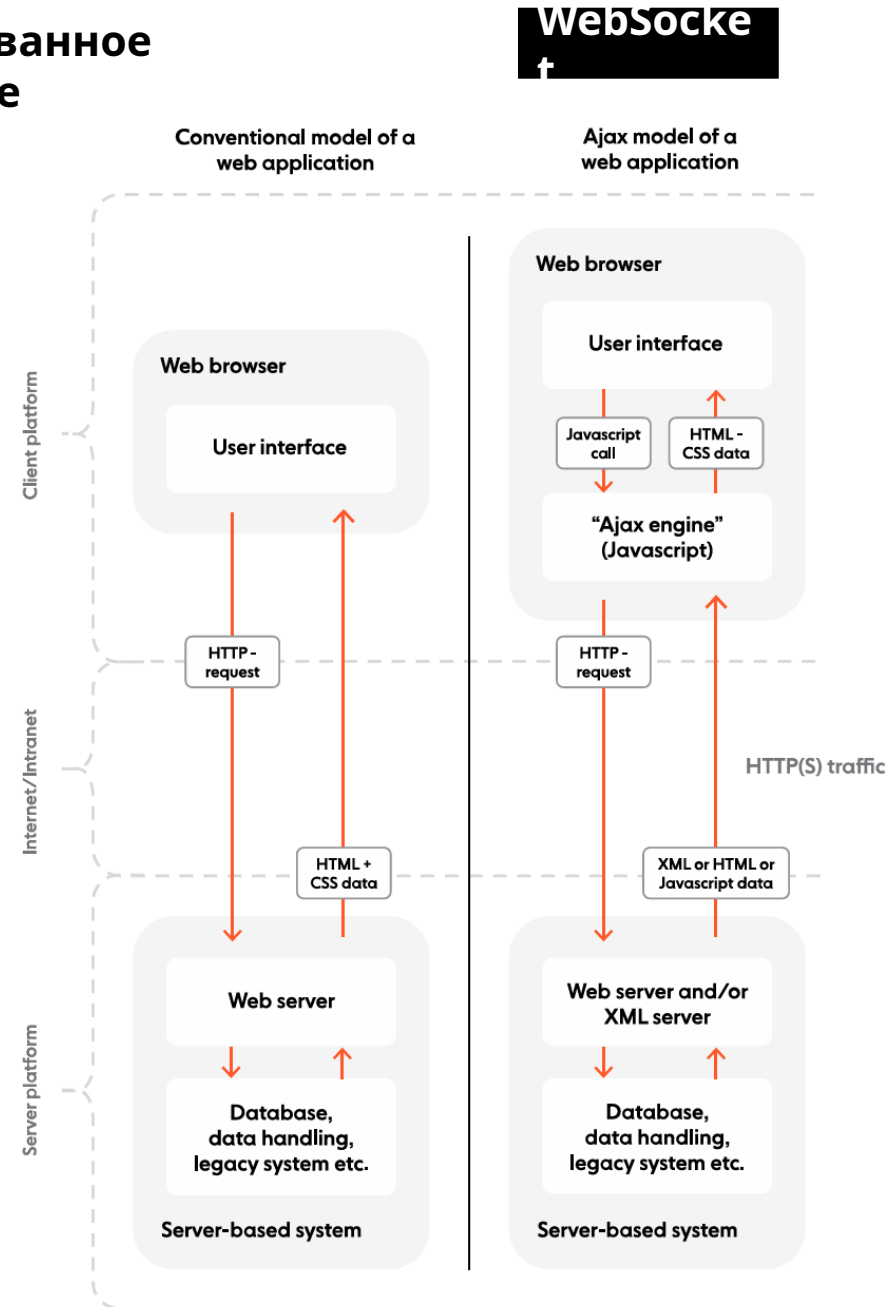
«Хакинг» HTML #1. AJAX

- AJAX (сокращение от Asynchronous JavaScript and XML) - это метод асинхронного обмена данными с сервером в фоновом режиме и обновления частей веб-страницы - без необходимости полного обновления страницы (постбэк).
- Впервые появился в 2005 году, AJAX использует следующие технологии:
 - HTML (или XHTML) и CSS для представления информации.
 - Объектную модель документа (DOM) для динамического отображения и взаимодействия.
 - XML или JSON для обмена данными
 - XSLT для манипуляций с XML.
 - XMLHttpRequest (XHR) для асинхронного взаимодействия.
 - JavaScript для связывания всего воедино.



«Хакинг» HTML #1. AJAX

Теперь давайте рассмотрим, как работает AJAX, сравнив его с классической моделью создания веб-приложения.



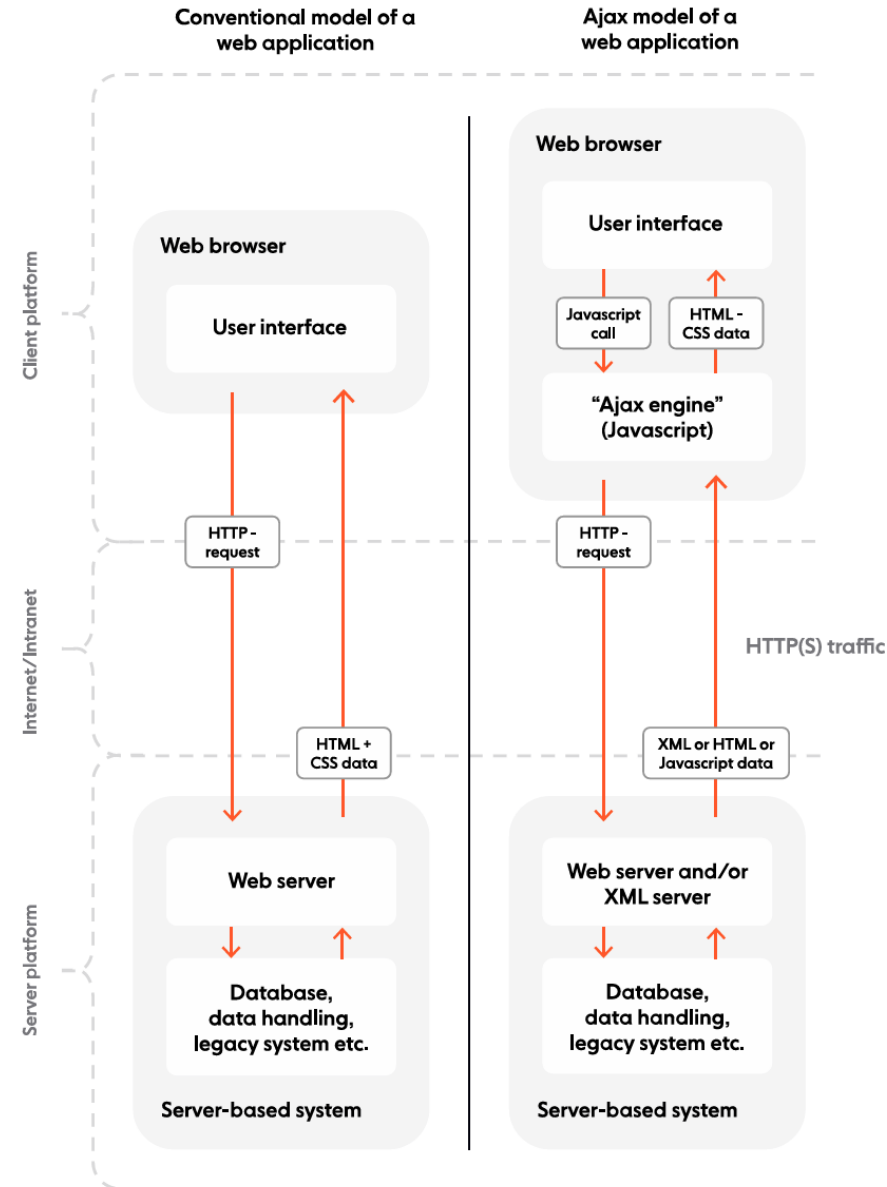
WebSocke
↑



«Хакинг» HTML #1. AJAX

- AJAX вводит посредника (движок AJAX) между пользователем и сервером, который значительно улучшает отзывчивость.
- Вместо загрузки веб-страницы в начале сеанса клиент загружает движок AJAX, который регулярно опрашивает сервер от имени клиента и обновляет DOM данными, полученными с сервера.

WebSocke
↑





«Хакинг» HTML #2. Comet

- Описанный в 2006 году, Comet - это модель дизайна веб-приложений, которая позволяет уже веб-серверу передавать данные браузеру.
- Comet обеспечивает асинхронное взаимодействие, но в отличие от классического AJAX (когда клиент периодически опрашивает сервер на предмет обновлений), Comet использует долговременные HTTP-соединения (periodically polls), что позволяет серверу отправлять обновления, когда они доступны, без явного запроса со стороны клиента.
- Comet изначально использовался в Gmail и потом стал стандартом по умолчанию для создания отзывчивых интерактивных веб-приложений, однако в последствии появились более эффективные реализации базовых технологий - long polling и HTTP streaming.



«Хакинг» HTML #2. Comet

- Описанный в 2006 году, Comet - это модель дизайна веб-приложений, которая позволяет уже веб-серверу передавать данные браузеру.
- Comet обеспечивает асинхронное взаимодействие, но в отличие от классического AJAX (когда клиент периодически опрашивает сервер на предмет обновлений), Comet использует долговременные HTTP-соединения (periodically polls), что позволяет серверу отправлять обновления, когда они доступны, без явного запроса со стороны клиента.
- Comet изначально использовался в Gmail и потом стал стандартом по умолчанию для создания отзывчивых интерактивных веб-приложений, однако в последствии появились более эффективные реализации базовых технологий - long polling и HTTP streaming.

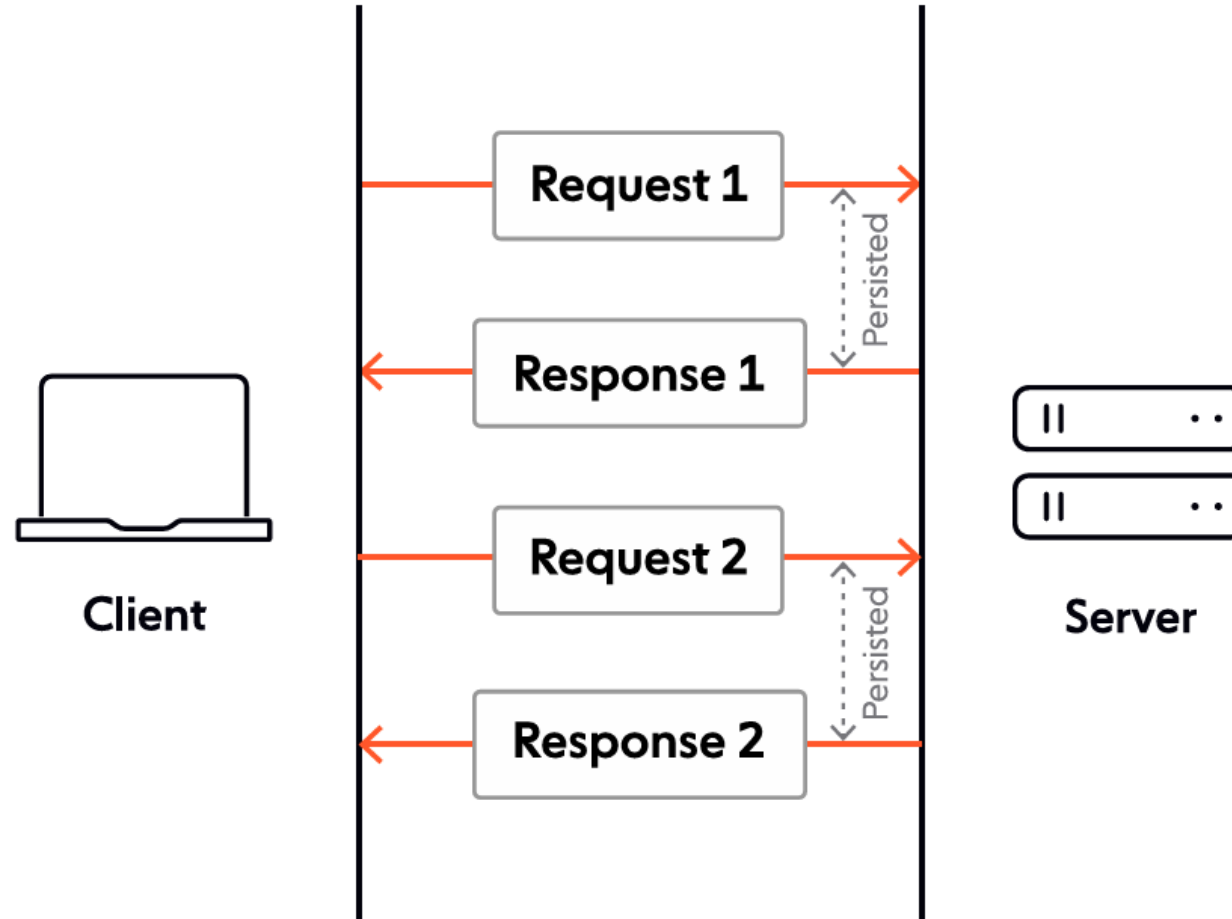


Long polling

- По сути, это более эффективная форма опроса.
- Длительный опрос - это техника, при которой сервер решает держать соединение с клиентом открытым как можно дольше, предоставляя ответ только после того, как данные станут доступны или будет достигнут порог таймаута.
- После получения ответа от сервера клиент обычно сразу же отправляет новый запрос.
- Длительный опрос часто реализуется на базе XMLHttpRequest, того самого объекта, который играет ключевую роль в модели AJAX.



Long polling





HTTP streaming

- Также известный как HTTP server push, HTTP streaming - это техника передачи данных, которая позволяет веб-серверу непрерывно отправлять данные клиенту через одно HTTP-соединение, которое остается открытым неограниченное время.
- При каждом обновлении сервер отправляет ответ и закрывает соединение только при явном указании сделать это. Поточковая передача HTTP может быть реализована с помощью механизма кодирования chunked transfer, доступного в HTTP/1.1. При таком подходе сервер может отправлять ответные данные в виде фрагментов строк, разделенных новой строкой, которые обрабатываются клиентом на лету.



HTTP streaming

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked
```

```
7\r\n
Chunked\r\n
8\r\n
Response\r\n
7\r\n
Example\r\n
0\r\n
\r\n
```



Server-Sent Events

- Server-Sent Events (SSE) - еще один вариант реализации потоковой передачи данных по HTTP. SSE - это технология серверного push, обычно используемая для отправки обновлений сообщений или непрерывных потоков данных клиенту браузера.
- SSE направлена на улучшение нативной, кроссбраузерной потоковой передачи данных от сервера к клиенту с помощью API JavaScript под названием EventSource, стандартизированного в рамках W3C.



Ограничения AJAX и Comet

- Большинство их ограничений связано с использованием HTTP в качестве основного транспортного протокола.
- Проблема в том, что HTTP изначально был разработан для обслуживания ресурсов в режиме запрос-ответ. Он не был оптимизирован для работы с приложениями реального времени, которые обычно предполагают высокочастотную или постоянную связь между клиентом и сервером, а также возможность мгновенно реагировать на изменения.
- «Хакинг» HTTP, для эмуляции веба в реальном времени неизбежно должен был привести к всевозможным недостаткам.



Ограничения AJAX и Comet

- Из ключевых:
 - Отсутствие гарантий на порядок сообщений
 - Задержки (медленно коннектимся + последовательно передаем)
 - Отсутствие двунаправленного потока

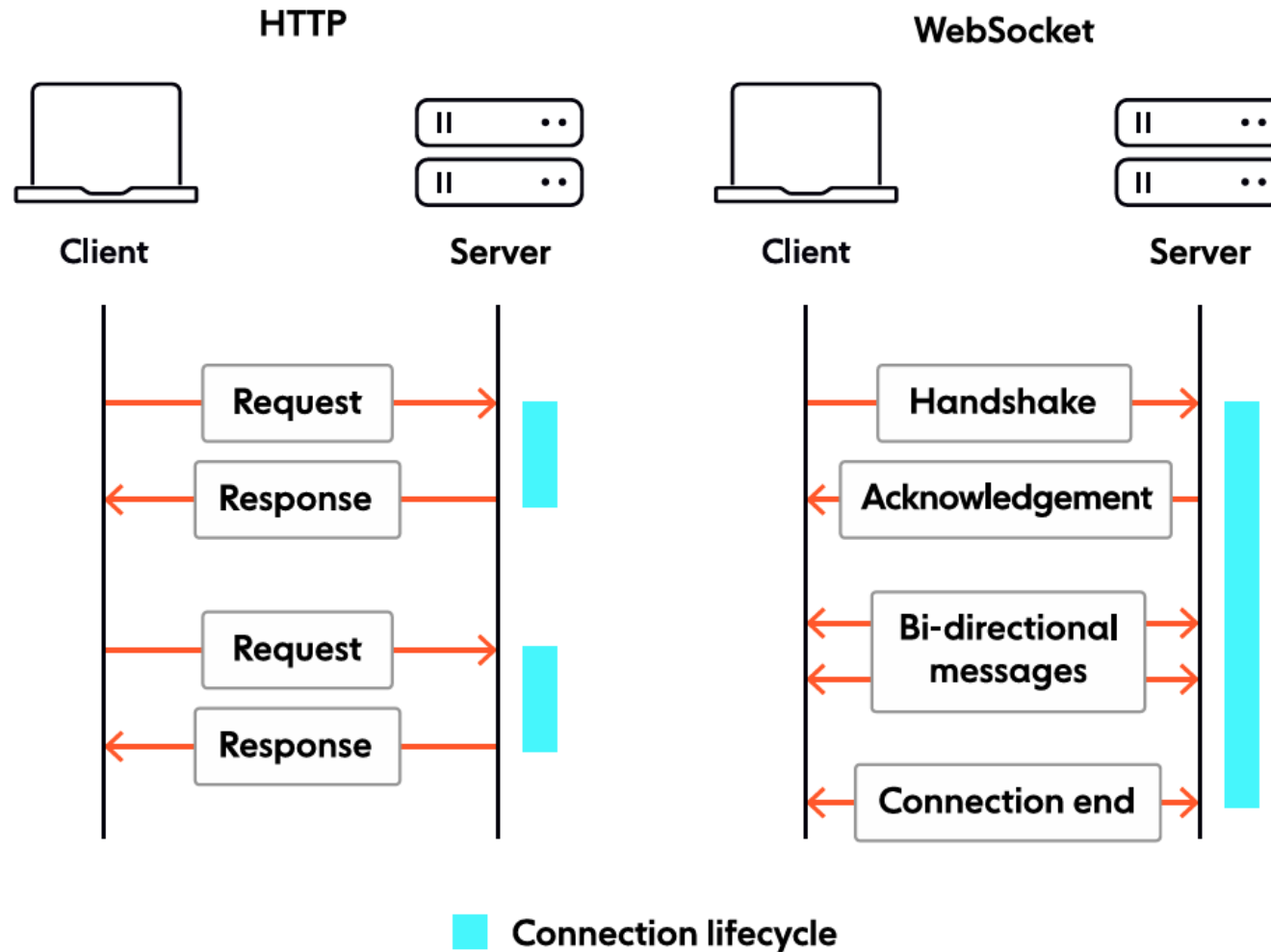


WebSocket

- Проще говоря, WebSocket - это технология, которая обеспечивает двунаправленную, полнодуплексную связь между клиентом и сервером через постоянное односокетное соединение.
- Цель состоит в том, чтобы предоставить разработчикам веб-приложений коммуникационный слой TCP, который, по сути, является близким к чистому, но при этом добавляет несколько абстракций, чтобы устранить некоторые проблемы, которые в противном случае существовали бы в связи с тем, как работает веб.
- Соединение WebSocket начинается как HTTP запрос/ответ;
 - за пределами этого рукопожатия, WebSocket и HTTP уже фундаментально отличаются.



WebSocket





Сравнение WebSocket и HTTP

- В то время как HTTP ориентирован на запросы, WebSockets ориентирован на события.

WEBSOCKETS	HTTP/1.1
Communication	
Full-duplex	Half-duplex
Message exchange pattern	
Bidirectional	Request-response
Server push	
Core feature	Not natively supported
Overhead	
Moderate overhead to establish the connection, and minimal overhead per message.	Moderate overhead per request/connection.
State	
Stateful	Stateless



Сценарии использования WebSocket

- Обновления в реальном времени, когда связь однонаправленная, и сервер передает клиенту обновления с низкой задержкой (и часто).
- Двухнаправленная связь, когда и клиент, и сервер могут отправлять и получать сообщения.



Преимущества использования WebSocket

- Повышенная производительность. По сравнению с HTTP, WebSockets избавляют от необходимости устанавливать новое соединение при каждом запросе, значительно уменьшая размер каждого сообщения (без HTTP-заголовков).
- Расширяемость. Гибкость заложена в конструкцию технологии WebSocket, что позволяет реализовывать подпротоколы (протоколы прикладного уровня) и расширения для дополнительной функциональности.
- Быстрое время реакции. Как технология, управляемая событиями, WebSockets позволяет передавать данные без запроса клиента. Эта характеристика желательна в сценариях, где клиенту необходимо быстро отреагировать на событие.



Протокол WebSocket

- Протокол WebSocket обеспечивает постоянную, полнодуплексную, двунаправленную связь между веб-серверами и веб-клиентами через базовое TCP-соединение.
- WebSocket состоит из начального рукопожатия (обновления соединения с HTTP до WebSockets), за которым следует передача данных.
 - После того, как клиент и сервер успешно согласуют начальное рукопожатие, соединение WebSocket действует как постоянный полнодуплексный канал связи, где каждая сторона может независимо отправлять данные по своему желанию.

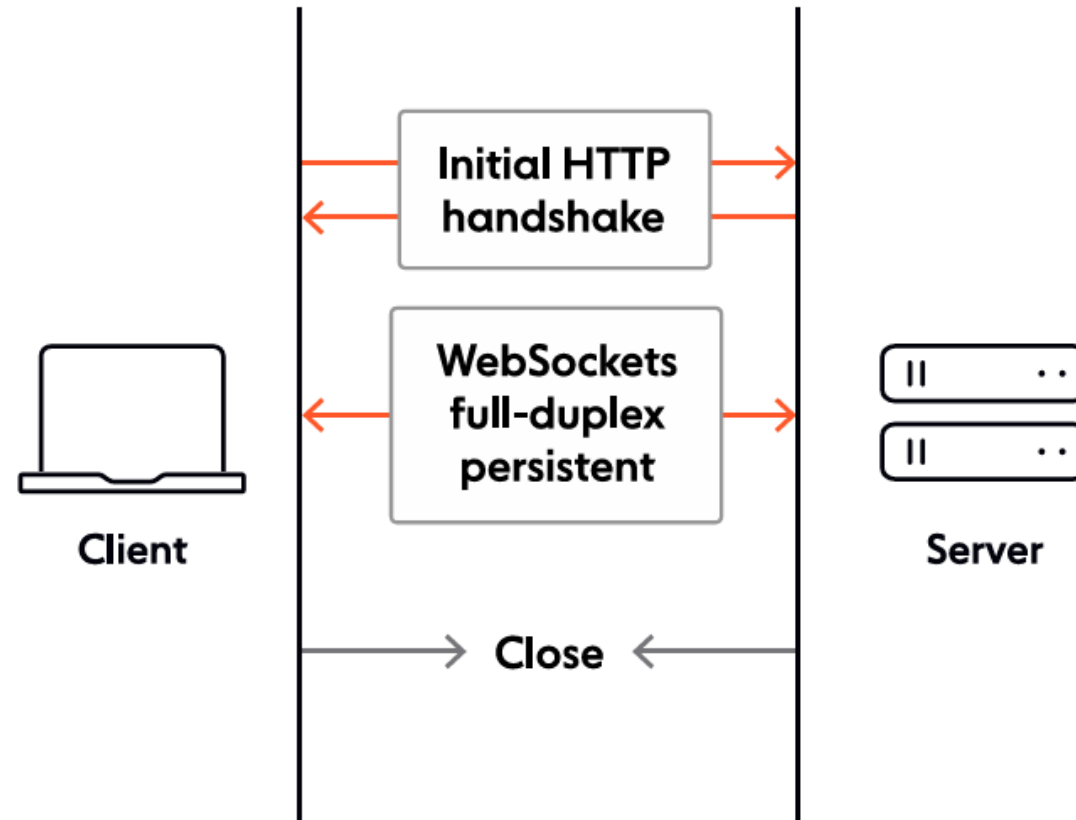


Протокол WebSocket

- Протокол WebSocket обеспечивает постоянную, полнодуплексную, двустороннюю связь между веб-серверами и веб-клиентами через базовое TCP-соединение.
- WebSocket состоит из начального рукопожатия (обновления соединения с HTTP до WebSockets), за которым следует передача данных.
 - Клиенты и серверы передают данные туда и обратно в концептуальных единицах, называемых сообщениями, которые, могут состоять из одного или нескольких кадров.
 - После того, как соединение WebSocket выполнило свою задачу, его можно завершить с помощью закрывающего рукопожатия.



Протокол WebSocket





Протокол WebSocket

- Протокол WebSocket определяет две схемы URI для трафика между сервером и клиентом:
 - ws, используемый для незашифрованных соединений.
 - wss, используемая для защищенных, зашифрованных соединений по протоколу Transport Layer Security (TLS).



Протокол WebSocket

- Протокол WebSocket определяет две схемы URI для трафика между сервером и клиентом:
 - Остальная часть URI WebSocket соответствует общему синтаксису, аналогичному HTTP. Он состоит из нескольких компонентов: хост, порт, путь и запрос, как показано в примере ниже.

`wss://example.com:443/websocket/demo?foo=bar`

Scheme Host Port Path Query



Протокол WebSocket

- Протокол WebSocket определяет две схемы URI для трафика между сервером и клиентом:
 - Стоит отметить, что:
 - Компонент порта является необязательным, по умолчанию используется порт 80 для ws и порт 443 для wss.
 - Идентификаторы фрагментов не допускаются в URI WebSocket.
 - Символ хэша (#) должен быть экранирован как %23



Процесс установления соединения

- Процесс установления соединения WebSocket называется открытием рукопожатия и состоит из обмена запросами/ответами HTTP/1.1 между клиентом и сервером.
- Клиент всегда инициирует рукопожатие
 - Он отправляет запрос GET на сервер, указывая, что он хочет обновить соединение с HTTP в WebSocket.
 - Сервер должен вернуть код ответа HTTP 101 Switching Protocols для установления соединения WebSocket.
 - После этого соединение WebSocket может использоваться для постоянной, двунаправленной, полнодуплексной связи между сервером и клиентом.



Запрос клиента

```
GET wss://example.com:8181/ HTTP/1.1
Host: localhost: 8181
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: zy6Dy9mSAIM7GJZNf9rI1A==
```

- Запрос должен содержать следующие заголовки:
 - Host
 - Connection
 - Upgrade
 - Sec-WebSocket-Version
 - Sec-WebSocket-Key



Ответ сервера

```
HTTP/1.1 101 Switching Protocols  
Connection: Upgrade  
Sec-WebSocket-Accept: EDJa7WCAQQzMCYNJM42Syuo9SqQ=  
Upgrade: websocket
```

- Ответ должен содержать несколько заголовков: Connection, Upgrade и Sec-WebSocket
- Асепт. Могут быть включены и другие необязательные заголовки, такие как Sec-WebSocket-Extensions или Sec-WebSocket-Protocol (при условии, что они были переданы в запросе клиента).



Ответ сервера

```
HTTP/1.1 101 Switching Protocols  
Connection: Upgrade  
Sec-WebSocket-Accept: EDJa7WCAQQzMCYNJM42Syuo9SqQ=  
Upgrade: websocket
```

- Если код состояния, возвращаемый сервером, не соответствует HTTP 101 Switching Protocol, рукопожатие не будет установлено и соединение WebSocket не будет установлено.



Подпротоколы

- Подпротоколы - это протоколы прикладного уровня, накладываемые поверх исходного протокола WebSocket.
 - Они помогают определить конкретные форматы и семантику более высокого уровня для обмена данными между клиентом и сервером.



Подпротоколы

- Message Queuing Telemetry Transport (MQTT)
- Simple Text Oriented Message Protocol (STOMP)
- и другие, даже пользовательские.

Stomp

The Simple Text Oriented Messaging Protocol



WebSocket API

- Сервер WebSocket может быть написан на любом языке программирования на стороне сервера, поддерживающем сокет Беркли.
- Сервер прослушивает входящие соединения WebSocket, используя стандартный сокет TCP.
- После согласования начального рукопожатия сервер должен иметь возможность отправлять, получать и обрабатывать сообщения WebSocket.



WebSocket API

- Чтобы начать работу с API WebSocket на стороне клиента, первое, что нужно сделать, — создать экземпляр объекта WebSocket, который автоматически попытается открыть соединение с сервером:

```
const socket = new WebSocket('wss://example.org');
```



WebSocket API

- Конструктор WebSocket содержит обязательный параметр — url к серверу WebSocket. Кроме того, может быть включен необязательный параметр protocols, чтобы указать один или несколько подпротоколов WebSocket (протоколов уровня приложения), которые могут использоваться во время клиент-серверного взаимодействия:

```
const socket = new WebSocket('wss://example.org', 'myCustomProtocol');
```



События

- Программирование WebSocket основано на асинхронной, событийно-ориентированной модели программирования.
- Пока соединение WebSocket открыто, клиент и сервер просто прослушивают события, чтобы обработать входящие данные и изменения в статусе соединения (без необходимости опроса).
 - open
 - message
 - error
 - close



Open

- Событие open поднимается, когда устанавливается соединение WebSocket. Оно указывает на то, что рукопожатие между клиентом и сервером прошло успешно, и теперь соединение WebSocket можно использовать для отправки и получения данных. Вот пример использования:

```
// Create WebSocket connection
const socket = new WebSocket('wss://example.org');

// Connection opened
socket.onopen = function(e) {
  console.log('Connection open!');
};
```



Message

- Событие сообщения запускается при получении данных через WebSocket. Сообщения могут содержать строку (обычный текст) или двоичные данные, и только от вас зависит, как эти данные будут обработаны и визуализированы. Вот пример того, как обрабатывать событие сообщения:

```
socket.onmessage = function(msg) {  
    if(msg.data instanceof ArrayBuffer) {  
        processArrayBuffer(msg.data);  
    } else {  
        processText(msg.data);  
    }  
}
```



Error

- Событие ошибки срабатывает в ответ на неожиданные сбои или проблемы (например, не удалось отправить некоторые данные). Вот как прослушивать события об ошибках

```
socket.onerror = function(e) {  
    console.log('WebSocket failure', e);  
    handleErrors(e);  
};
```



Close

- Событие close срабатывает, когда соединение WebSocket закрывается. Это может произойти по разным причинам, например, из-за сбоя соединения, успешного завершения квитирования или ошибок TCP. Когда соединение прерывается, клиент и сервер больше не могут отправлять или получать сообщения. Вот как можно прослушать событие закрытия:

```
socket.onclose = function(e) {  
    console.log('Connection closed', e);  
};
```



Методы

- send()
- После того как соединение установлено, вы почти готовы начать отправлять и получать сообщения на сервер WebSocket. Но перед этим необходимо убедиться, что соединение открыто и готово к приему сообщений. Этого можно добиться двумя основными способами.

```
socket.onopen = function(e) {  
    socket.send(JSON.stringify({'msg': 'payload'}));  
}
```



Методы

- send()
- После того как соединение установлено, вы почти готовы начать отправлять и получать сообщения на сервер WebSocket. Но перед этим необходимо убедиться, что соединение открыто и готово к приему сообщений. Этого можно добиться двумя основными способами.

```
function processEvent(e) {  
  if(socket.readyState === WebSocket.OPEN) {  
    // Socket open, send!  
    socket.send(e);  
  } else {  
    // Show an error, queue it for sending later, etc  
  }  
}
```



Методы

- В двух приведенных выше фрагментах кода показано, как отправлять текстовые (строковые) сообщения. Однако, помимо строк, можно отправлять и двоичные данные (Blob или ArrayBuffer), как показано в этом примере

```
var buffer = new ArrayBuffer(128);  
socket.send(buffer);  
  
var intview = new Uint32Array(buffer);  
socket.send(intview);  
  
var blob = new Blob([buffer]);  
socket.send(blob);
```



Методы

- close()
- Метод close() используется для закрытия соединения WebSocket (или попытки соединения).

```
socket.close();
```



Методы

- По желанию можно передать два аргумента методу `close()`:
 - Код. Числовое значение, указывающее код состояния, объясняющий причину закрытия соединения.
 - Причина. Человекочитаемая строка, объясняющая причину закрытия соединения.

```
socket.close(1003, 'Unsupported data type!');
```



Свойства

- Объект WebSocket содержит несколько свойств, содержащих подробную информацию о соединении WebSocket.
 - `binaryType`
 - Свойство `binaryType` управляет типом двоичных данных, получаемых через соединение WebSocket. По умолчанию используется тип `blob`.
 - `bufferedAmount`
 - Свойство только для чтения, которое возвращает количество байт данных, поставленных в очередь на передачу, но еще не отправленных. Значение `bufferedAmount` обнуляется после того, как все данные, поставленные в очередь, будут отправлены.



Свойства

- Ниже приведен пример использования `bufferedAmount` для отправки обновлений каждую секунду и соответствующей корректировки, если сеть не справляется с такой скоростью.

```
// 10k max buffer size.
var THRESHOLD = 10240;

// Create a New WebSocket connection
const socket = new WebSocket('wss://example.org');

// Listen for the opening event
socket.onopen = function () {
    // Attempt to send update every second.
    setInterval(function () {
        // Send only if the buffer is not full
        if (socket.bufferedAmount < THRESHOLD) {
            socket.send(getApplicationState());
        }
    }, 1000);
};
```



Свойства

- `onevent`
- Эти свойства вызываются для выполнения соответствующего кода обработчика при наступлении события WebSocket. Существует четыре типа свойств «onevent», по одному для каждого типа события:

PROPERTY	DESCRIPTION
<code>onopen</code>	Called when the WebSocket connection's <code>readyState</code> property changes to 1; this indicates that the connection is open and ready to send and receive data.
<code>onmessage</code>	Called when a message is received from the server.
<code>onerror</code>	Gets called when an error event occurs, impacting the WebSocket connection.
<code>onclose</code>	Called with a close event when the WebSocket's connection <code>readyState</code> property changes to 3; this indicates that the connection is closed.



Свойства

- readyState
- Свойство, доступное только для чтения, которое возвращает текущее состояние соединения WebSocket.

VALUE	STATE	DESCRIPTION
0	CONNECTING	Socket has been created, but the connection is not yet open.
1	OPEN	The connection is open and ready to communicate.
2	CLOSING	The connection is in the process of closing.
3	CLOSED	The connection is closed or couldn't be opened.



Свойства

- url
- Свойство, доступное только для чтения, которое возвращает абсолютный URL-адрес WebSocket, определенный конструктором



Будут вопросы? Не стесняйтесь!

